

Міністерство освіти і науки України
Національний університет "Львівська політехніка"



Звіт до лабораторної роботи №5
З дисципліни "Web-технології та Web-дизайн"
на тему: "CRUD Javascript App: Backend"

Виконала:

Ст.гр. ІР-11

Луцан І.З.

Прийняв:

доцент, к.т.н.

Іванюк О.О.

Львів 2025

Завдання:

5. CRUD Javascript App: Backend

Description: In the last part of working on the website you have to implement all Create/Read/Update/Delete operations which must be made via the corresponding HTTP methods in your REST API.

Important. You don't have to make a backend from the beginning - connect an existing one that you worked on in the first year.
If you don't have a backend server, you should create a new REST API using any preferred technology.

Guide for creating Node.js server, if you would like also to try JavaScript as a backend language >>> [\[LINK\]](#)

Our recommendations and tips:

- [Working with REST API with fetch\(\) link](#) (POST method example)
- a website [example](#) from live coding

Реалізація бек-енду:

файл app.py

```
import os

from flask import Flask, request, jsonify, render_template

from flask_sqlalchemy import SQLAlchemy

from flask_cors import CORS


# Ініціалізація Flask
# static_folder='static' and template_folder='templates' are default,
# but added for clarity.

app = Flask(__name__, template_folder='templates', static_folder='static')

CORS(app)


basedir = os.path.abspath(os.path.dirname(__file__))

app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:/// ' + os.path.join(basedir, 'shoes.db')

app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
```

```
db = SQLAlchemy(app)

@app.route("/")
def index():

    """Serves the main index.html page."""

    return render_template("index.html")

@app.route("/create")
def create():

    """Serves the create.html page."""

    return render_template("create.html")

@app.route("/edit")
def edit():

    """Serves the edit.html page."""

    return render_template("edit.html")

# --- Модель даних (Таблиця) ---

class Shoe(db.Model):

    id = db.Column(db.Integer, primary_key=True)

    producer = db.Column(db.String(100), nullable=False)

    price = db.Column(db.Float, nullable=False)

    size = db.Column(db.Integer, nullable=False)

    color = db.Column(db.String(50), nullable=False)

    # (перетворення об'єкта Python в JSON)

    def to_dict(self):

        return {

            'id': self.id,

            'producer': self.producer,

            'price': self.price,

            'size': self.size,
```

```

        'color': self.color

    }

# CRUD Операції (API Endpoints)

# CREATE (Створити)

@app.route('/api/shoes', methods=['POST'])

def add_shoe():

    data = request.get_json()

    if not all(k in data for k in ('producer', 'price', 'size', 'color')):

        return jsonify({'error': 'Missing data'}), 400

    try:

        new_shoe = Shoe(

            producer=data['producer'],

            price=float(data['price']),

            size=int(data['size']),

            color=data['color']

        )

        db.session.add(new_shoe)

        db.session.commit()

        return jsonify(new_shoe.to_dict()), 201

    except (ValueError, TypeError):

        return jsonify({'error': 'Invalid data format'}), 400

# READ (Отримати всі)

@app.route('/api/shoes', methods=['GET'])

def get_shoes():

    shoes = Shoe.query.all()

    return jsonify([shoe.to_dict() for shoe in shoes])

```

```
# READ

@app.route('/api/shoes/<int:id>', methods=['GET'])

def get_shoe(id):

    shoe = db.session.get(Shoe, id)

    if shoe:

        return jsonify(shoe.to_dict())

    else:

        return jsonify({'error': 'Shoe not found'}), 404


# UPDATE

@app.route('/api/shoes/<int:id>', methods=['PUT'])

def update_shoe(id):

    shoe = db.session.get(Shoe, id)

    if not shoe:

        return jsonify({'error': 'Shoe not found'}), 404

    data = request.get_json()

    try:

        shoe.producer = data.get('producer', shoe.producer)

        shoe.price = float(data.get('price', shoe.price))

        shoe.size = int(data.get('size', shoe.size))

        shoe.color = data.get('color', shoe.color)

        db.session.commit()

        return jsonify(shoe.to_dict())

    except (ValueError, TypeError):

        return jsonify({'error': 'Invalid data format'}), 400


# DELETE

@app.route('/api/shoes/<int:id>', methods=['DELETE'])

def delete_shoe(id):
```

```

shoe = db.session.get(Shoe, id)

if shoe:

    db.session.delete(shoe)

    db.session.commit()

    return jsonify({'message': 'Shoe deleted successfully'})

else:

    return jsonify({'error': 'Shoe not found'}), 404

# --- Запуск сервера ---

if __name__ == '__main__':

    # Створюємо таблиці, якщо їх ще немає

    with app.app_context():

        db.create_all()

    # Запускаємо Flask-сервер

    # debug=True автоматично перезавантажує сервер при змінах

    app.run(port=5001, debug=True)

```

Посилання на лабораторну роботу на гітхабі:

https://github.com/IrynaLutsan/Web_Programming_Labs/tree/main/Lab_5

Висновок:

У ході лабораторної роботи було реалізовано бекенд для вебзастосунку з підтримкою CRUD-операцій через REST API. Було налаштовано обробку запитів для створення, читання, оновлення та видалення даних із використанням відповідних HTTP-методів. Для цього можна було застосувати як уже наявний сервер, так і створити новий, використавши будь-яку зручну технологію. Робота дозволила закріпити навички взаємодії фронтенду з бекендом та організації коректної структури серверної частини застосунку.